Rho restart fix: v2 IC4 re-measured (2026-11-11)

Correction results: v2 IC4 vs the fixed rho reference

Protocol: `PROTOCOL.md` (frozen in 75dbc70b36 before data). Rerun: every rho-family job of the v2 holdout,
1584 runs with the fixed binary, **0 errors, every target verified**. `results/rows.jsonl` (new rho rows),
`results/merged.jsonl` (v2 `ic4` and `bsgs2` rows + new rho rows), `results/analysis.txt` (v2 `analyze.py`
unchanged), `results/summary.json`. All numbers [pcc:unverified] (claim PCC-20261111-e5ded0, superseding PCC-20261111-fa13c8 for its rho comparison) until the claim is s
`validation/VALIDATOR_REPORT.md` (valid_with_amendments: every merged row replays exactly, every number recomputes;
the "survives" headline is insufficient evidence against a standard rho, applied below).

IC4 / rhobest (median per-configuration ratio, range over c_I in {3, 10, 30} and h in {0, 0.1, 1})

bits	T	v2 as published	corrected	v2 decision (all c_I)	corrected decision (c_I = 3 / 10 / 30)
22	1024	1.053-1.122	1.074-1.133	dearer	dearer / dearer / dearer
24	1024	1.094-1.188	1.123-1.210	dearer	dearer / dearer / dearer
24	4096	0.961-1.021	0.985-1.038	unranked	unranked / unranked / dearer
24	16384	0.918-0.949	0.951-0.984	**cheaper**	**cheaper / cheaper / cheaper**
25	4096	0.972-1.038	0.997-1.057	unranked	unranked / dearer / dearer
25	16384	0.908-0.944	0.951-0.991	**cheaper**	**cheaper / cheaper / cheaper**

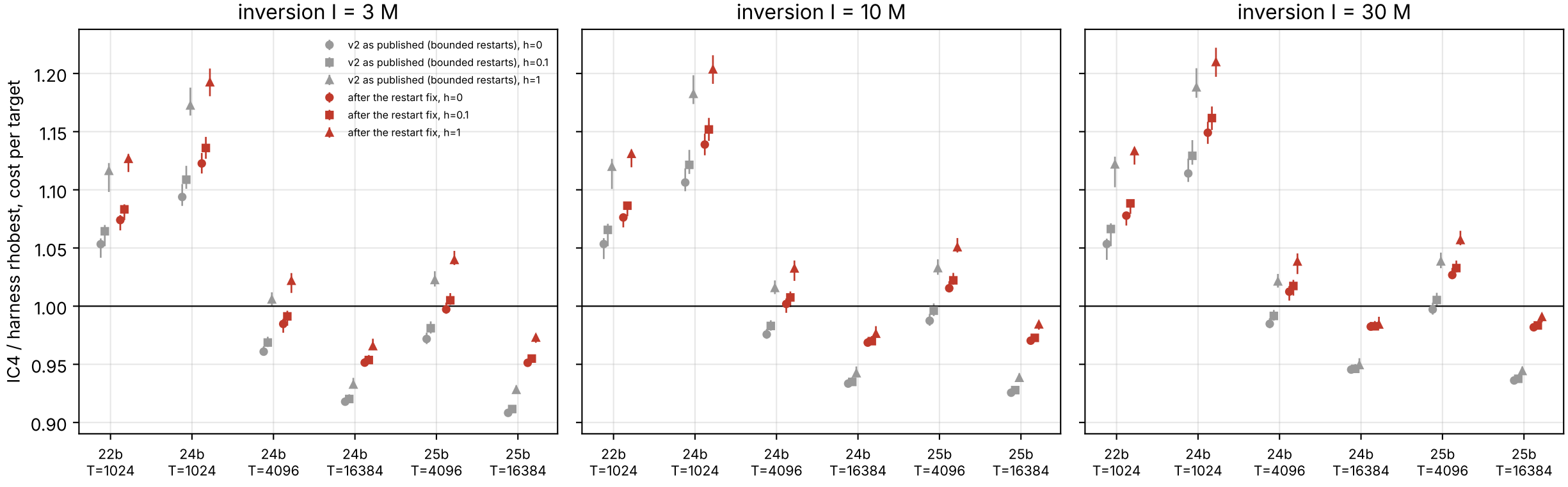
IC4 / bsgs2 is unchanged (1.18-1.80; neither arm's code changed).

What changed and what did not

- The defect made the rho reference 1.4% to 4.8% too expensive in the cheapest-arm cost (per cell, c_I and h). The
  fix is not monotone per run: 3.5% to 15.7% of runs got dearer, by up to 4.3%, and chained starts repeat a walk
  with probability 1/64 per restart.
- **Within this harness's rho family** IC4 stays cheaper at T = 16384 for every c_I and h, by 1.6% to 4.9% (median
  ratio 0.951-0.991), not 5% to 9%; at 25 b, c_I = 30, h = 1 the margin is 0.9%. At T = 4096 three (cell, c_I)
  decisions move from unranked to dearer.
- **That is not robust.** Removing two avoidable overheads of the harness rho (restart cost, repeated lookups) leaves
  IC4 cheaper at T = 16384 only at c_I = 3. With Kuhn-Struik retention of finished walks added to rho2x (validator,
  528 runs, every target verified [pcc:unverified]), IC4 at T = 16384 is 1.12x-1.17x that rho at h = 0, and the v2 decisions become
  0 cheaper, 6 unranked, 12 dearer. The v2 claim that IC4 is cheaper than rho at T = 16384 is therefore withdrawn as a
  statement about rho; it holds only against this harness's anchored, fixed-database rho family.
- The pre-registered prediction (T = 16384 moves to about 1.0, unranked) was wrong for the harness family (the
  margin halved but stayed below 1 at every h).
- IC4 / bsgs2 is unchanged (1.18-1.80; neither arm's code changed).

Consequences (applied in the PR that lands this file)

- Claim PCC-20261111-fa13c8 (v2) is superseded by the claim recorded with this correction.
- `benchmarks/20261111-mt-fieldops-ic4/` (README, VALIDATION.json: `strongest_available` false) and its row in
  `benchmarks/README.md` carry the corrected range and the retention result; the leaderboard's multi-target section
  moves to the v3 rows (`../mt_fieldops_v3_20261111/`), which carry the same caveat.
- `rows.jsonl` and `merged.jsonl` are moved to the artifact store by the store bot when the PR opens
  (`results/runs.json` keeps the merged counters in Git).


inversion I = 3 M
inversion I = 10 M
inversion I = 30 M
IC4 / harness rhobest, cost per target
v2 as published (bounded restarts), h=0
v2 as published (bounded restarts), h=0.1
v2 as published (bounded restarts), h=1
after the restart fix, h=0
after the restart fix, h=0.1
after the restart fix, h=1
1.20
1.15
1.10
1.05
1.00
0.95
0.90
22b T=1024
24b T=1024
24b T=4096
24b T=16384
25b T=4096
25b T=16384